

Resumen completo

TEMA 1: Arquitectura de Aplicaciones Empresariales

Requisitos de Aplicaciones Empresariales

- **Escalabilidad:** Capacidad de soportar más usuarios/carga al aumentar recursos
- **Alta disponibilidad:** Tolerancia a fallos mediante replicación
- **Transaccionalidad:** Propiedades ACID
- **Seguridad:** Diferentes niveles de acceso por usuario

Diseño por Capas

Capa	Función
Interfaz Gráfica	Presenta la funcionalidad al usuario
Servicios	API remota para otras aplicaciones
Lógica de Negocio	Implementa los casos de uso
Acceso a Datos	Gestiona persistencia (BD, otras apps)

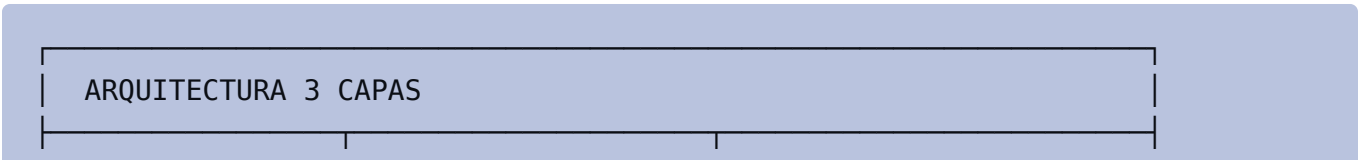
Ventajas del diseño por capas:

- Desarrollo en paralelo de capas
- Facilita mantenimiento (cambios en una capa no afectan otras)
- Facilita escalabilidad y tolerancia a fallos
- Desarrolladores no necesitan conocer tecnologías de otras capas

 Important

Pregunta típica: La arquitectura de 2 capas (cliente con Modelo + servidor BD) vs 3 capas: La desventaja de 2 capas es que **cambios en la capa Modelo requieren reinstalar en máquinas cliente.**

Distribución de Capas Típicas



Capa 1: Clientes (IU + Acceso a Servicios)	Capa 2: Servidor Apps (Servicios + Modelo)	Capa 3: Servidor Datos (Base de Datos)
---	---	--

Tecnologías Java

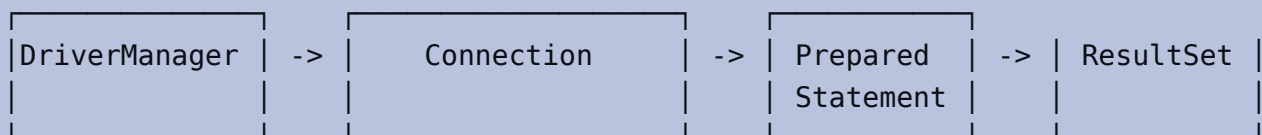
- **Java SE:** API básica
- **Java EE/Jakarta EE:** APIs para aplicaciones empresariales
- **JDBC:** Acceso a BD relacionales
- **Servlets:** Capa interfaz web
- **REST/Thrift:** Capa servicios

TEMA 2: JDBC (Java DataBase Connectivity)

Conceptos Fundamentales

- API estándar para acceso a BD relacionales
- Paquetes: `java.sql` y `javax.sql`
- Independiente del driver específico de la BD

Principales Interfaces



PreparedStatement

🔥 Important

Pregunta típica: `executeQuery()` permite consultas de **lectura (SELECT)** y siempre devuelve un `ResultSet`. `executeUpdate()` permite actualización (INSERT, UPDATE, DELETE) y devuelve el número de filas afectadas.

```
// Parámetros se numeran desde 1
preparedStatement.setString(1, valor);
```

```
preparedStatement.setLong(2, id);
```

Liberación de Recursos

```
// USO CORRECTO con try-with-resources
try (Connection connection = dataSource.getConnection()) {
    // operaciones con BD
} // connection se cierra automáticamente
```

Caution

Siempre cerrar conexiones (explícitamente o con try-with-resources). Si no se cierran, en aplicaciones multi-thread se agotarán las conexiones disponibles.

Pool de Conexiones

- DataSource puede implementar pool de conexiones
- `getConnection()` devuelve conexión del pool (rápido)
- `close()` **NO cierra la conexión real**, la devuelve al pool

Important

Pregunta típica: Al invocar `close()` sobre una conexión del pool, **la conexión contra la BD no se cierra** (se devuelve al pool).

Transacciones

```
connection.setTransactionIsolation(Connection.TRANSACTION_SERIALIZABLE);
connection.setAutoCommit(false);
// ... operaciones ...
connection.commit(); // o connection.rollback();
```

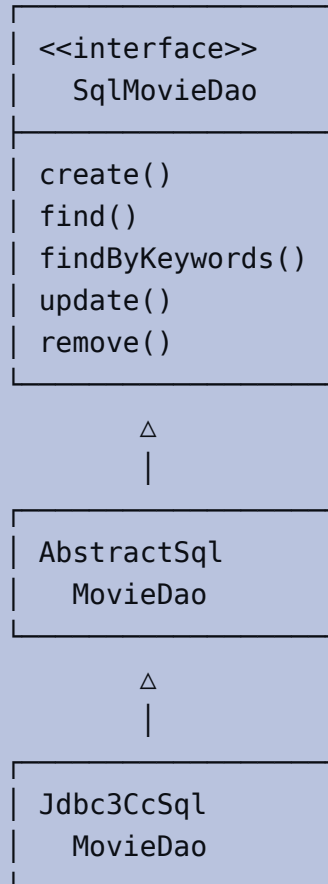
TEMA 3: Diseño e Implementación de la Capa Modelo

Método de Desarrollo

1. **Modelar entidades** (clases persistentes)
2. **Definir API** (interfaces Facade)

3. **Crear DAOs** para persistencia
4. **Implementar casos de uso**
5. **Implementar pruebas de integración**

Patrón DAO (Data Access Object)



Important

Pregunta típica: Los DAOs que reciben `Connection` como parámetro **NO ocultan** el tipo de BD (relacional) ni la tecnología de acceso (JDBC). Las operaciones del DAO **NO deben declarar `InputValidationException`**.

Generación de Claves Numéricas

BD con Secuencias (Oracle)	BD con Columnas Contador (MySQL)
Generar ID primero con consulta especial	Insertar fila sin ID
Luego insertar con el ID	Usar <code>RETURN_GENERATED_KEYS</code> para obtener ID

```
// MySQL - columnas contador con JDBC 3.0+
PreparedStatement ps = connection.prepareStatement(
    queryString, Statement.RETURN_GENERATED_KEYS);
ps.executeUpdate();
ResultSet rs = ps.getGeneratedKeys();
Long id = rs.getLong(1);
```

Gestión de Transacciones en Capa Modelo

Caso de uso con solo lectura:

- Modo auto-commit
- Nivel de aislamiento por defecto

Caso de uso con modificaciones:

1. Validar datos de entrada
2. `setTransactionIsolation(TRANSACTION_SERIALIZABLE)`
3. `setAutoCommit(false)`
4. Ejecutar lógica con DAOs
5. Si error de negocio: `commit()` + excepción checked
6. Si `SQLException/RuntimeIOException`: `rollback()` + relanzar

Important

Pregunta típica: Si se viola una regla de negocio (antes de modificar), se invoca `commit()` y se lanza excepción. Si ocurre `SQLException`, se invoca `rollback()`.

Factorías y Singletons

```
// Obtención de referencia a servicio
MovieService service = MovieServiceFactory.getService();

// Obtención de DAO
SqlMovieDao dao = SqlMovieDaoFactory.getDao();

// Configuración en ConfigurationParameters.properties
MovieServiceFactory.className=...MovieServiceImpl
SqlMovieDaoFactory.className=...Jdbc3CcSqlMovieDao
```

TEMA 4: Pruebas de Integración

Conceptos Clave JUnit 5

- `@BeforeAll`: Método ejecutado una vez antes de todos los tests
- `@Test`: Marca un método como caso de prueba
- `assertEquals()`, `assertThrows()`: Aserciones

Independencia entre Casos de Prueba

Important

Cada caso de prueba debe:

1. **Crear los datos que necesita**
2. Invocar el caso de uso
3. Verificar resultados
4. **Eliminar los datos creados**

Pregunta típica: Para crear datos, usar métodos del servicio de la capa Modelo **si existen**. Para eliminar, usar DAOs directamente si no hay método en el servicio.

DataSourceLocator

```
// En pruebas (@BeforeAll):
DataSource dataSource = new SimpleDataSource();
DataSourceLocator.addDataSource("ws-javaexamples-ds", dataSource);

// En implementación del servicio:
dataSource = DataSourceLocator.getDataSource("ws-javaexamples-ds");
```

- **Pruebas:** SimpleDataSource (sin pool, lee de `SimpleDataSource.properties`)
- **Servidor de aplicaciones:** DataSource por JNDI (con pool)

Configuración de Pruebas

- Fichero: `src/test/resources/SimpleDataSource.properties`
- BD de pruebas: `wstest` (diferente a `ws` de desarrollo)

Métodos Privados de Ayuda

```
// Ejemplo: relanzar excepciones checked como RuntimeException
private void removeProduct(Long id) {
    try {
        productService.removeProduct(id);
    } catch (InstanceNotFoundException e) {
        throw new RuntimeException(e);
    }
}
```

TEMA 5: XML y JSON

XML - Reglas Fundamentales

- Tags se abren `<tag>` y cierran `</tag>`
- Documento debe tener **un único elemento raíz**
- Atributos van entrecomillados: `attr="value"`
- Elementos vacíos: `<tag attr="val"/>`

Entidades predefinidas:

| Referencia | Carácter |

|-----|-----|

| `<` | `<` |

| `>` | `>` |

| `&` | `&` |

| `"` | `"` |

| `'` | `'` |

Espacios de Nombres XML

```
<movies xmlns="http://ws.udc.es/movies/xml"
        xmlns:review="http://reviews.example.com">
```

Parsers XML

- **DOM**: Construye árbol en memoria (fácil de usar, más memoria)
- **Streaming (SAX)**: Procesa secuencialmente (menos memoria, más complejo)

JSON - Estructura

```
{
  "nombre": "valor string",
  "numero": 123,
  "booleano": true,
  "nulo": null,
  "array": [1, 2, 3],
  "objeto": {"clave": "valor"}
}
```

Important

Pregunta típica de JSON:

```
[{"title": "Learning Java", "formats": ["PDF", "EPUB"], "price": 1}]
```

Es un **array** con un único elemento tipo **objeto**, con tres campos: string, array y número.

Jackson (Parser JSON)

```
// Parsing
ObjectMapper mapper = new ObjectMapper();
JsonNode root = mapper.readTree(inputStream);

// Acceso
String title = root.get("title").textValue();
long id = root.get("movieId").longValue();

// Generación
ObjectNode node = JsonNodeFactory.instance.objectNode();
node.put("title", "valor").put("price", 9.99);
```

Validación

Note

Una aplicación que parsee XML/JSON **siempre** comprueba que esté **bien formado**. La validación contra esquema es **opcional**.

Ventajas de NO validar estrictamente:

- Eficiencia
 - Permite evolución (añadir campos sin romper clientes antiguos)
-

TEMA 6: Servicios Web REST

Principios REST

- **Stateless:** Servidor no guarda estado de cliente
- **Recursos identificados por URLs**
- **Interfaz uniforme:** GET, POST, PUT, DELETE
- **Representaciones:** JSON/XML de los recursos

Métodos HTTP

Método	Uso	Seguro	Idempotente
GET	Obtener representación	✓	✓
POST	Crear recurso / acciones	✗	✗
PUT	Reemplazar representación	✗	✓
DELETE	Eliminar recurso	✗	✓

Códigos de Respuesta HTTP

Código	Significado	Uso en servicio
200 OK	Éxito	GET exitoso
201 Created	Recurso creado	POST exitoso
204 No Content	Éxito sin cuerpo	PUT/DELETE exitoso
400 Bad Request	Error cliente	InputValidationException
403 Forbidden	Prohibido	MovieNotRemovableException
404 Not Found	No existe	InstanceNotFoundException
410 Gone	Ya no existe	SaleExpirationException
500 Internal Error	Error servidor	Errores de infraestructura

Pregunta típica: Para inscribirse en una carrera (crear inscripción), hacer **POST** a **/inscripciones** con los datos como parámetros.

Diseño de Recursos - Ejemplo Movies

GET /movies?keywords=xxx	→ Lista de películas
POST /movies	→ Crear película
PUT /movies/{id}	→ Actualizar película
DELETE /movies/{id}	→ Eliminar película
POST /sales	→ Comprar película (crear venta)
GET /sales/{id}	→ Obtener venta

Servlets

```
public class MoviesServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) {
        String keywords = req.getParameter("keywords");
        // ... procesar y responder
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
    {
        // Los parámetros en POST también se obtienen con getParameter()
    }
}
```

Note

`getPathInfo()` devuelve la parte del path después del patrón del servlet.

- `/movies` → `null`
- `/movies/123` → `/123`

Capa Acceso a Servicios (Cliente REST)

```
// Usando HttpClient Fluent API
ClassicHttpResponse response = (ClassicHttpResponse)
```

```
Request.get(url + "?keywords=" + URLEncoder.encode(keywords, "UTF-8"))
    .execute().getResponse();

int statusCode = response.getStatusCode();
InputStream body = response.getEntity().getContent();
```

Intermediarios REST

- **Cache:** Cachea respuestas GET exitosas (200 OK)
- **Seguridad:** Control de acceso transparente
- **Traducción de formatos:** XML ↔ JSON

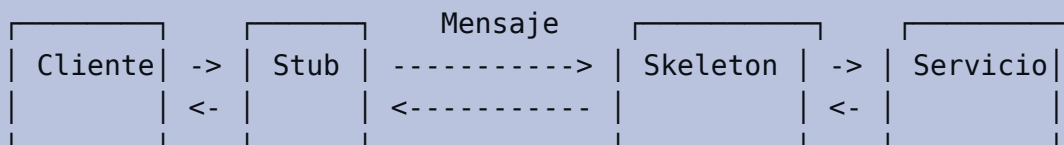
Important

Pregunta típica de cache: Es correcto cachear GET que devuelve 200 OK. Es correcto invalidar cache de recurso tras DELETE exitoso (204).

TEMA 7: Apache Thrift (RPC)

Modelo RPC

1. Desarrollador define interfaz en **fichero IDL**
2. Compilador genera **stub** (cliente) y **skeleton** (servidor)
3. Cliente invoca stub como librería local
4. Framework gestiona comunicación transparentemente



IDL de Apache Thrift

Tipos base:

```
| IDL | Java |
|-----|
| i16 | short |
| i32 | int   |
| i64 | long  |
```

double	double
string	String
bool	boolean

Estructuras:

```
namespace java es.udc.ws.movies.thrift

struct ThriftMovieDto {
    1: i64 movieId
    2: string title
    3: i16 runtime
}

exception ThriftInstanceNotFoundException {
    1: string instanceId
    2: string instanceType
}

service ThriftMovieService {
    ThriftMovieDto addMovie(1: ThriftMovieDto movieDto)
        throws (1: ThriftInputValidationException e)

    list<ThriftMovieDto> findMovies(1: string keywords)
}
```

Important

Pregunta típica: Un `struct` NO puede usarse como tipo de excepción. Las excepciones se declaran con `exception`.

Evolución de Interfaces Thrift

- Se puede **cambiar nombre** de campos (se identifica por ID)
- Se pueden **añadir/eliminar** campos
- Si se elimina, **no reutilizar el ID**

TEMA 8: Aplicaciones Web

Aplicaciones Web del Lado Servidor

- HTML se genera en el servidor
- Se instalan en servidor de aplicaciones (WAR)
- Cada petición HTTP genera una página completa

Aplicaciones SPA (Single Page Application)

- Capa IU en **JavaScript** ejecutándose en navegador
- Backend: Servicio REST/JSON
- Frontend: Modifica DOM dinámicamente
- Modelo **asíncrono** (AJAX)

```
fetch('http://.../movies?keywords=matrix')
  .then(response => /* procesar */)
  .catch(error => /* manejar error */);
```

Comparación

Aspecto	Lado Servidor	SPA
HTML generado en	Servidor	Cliente (JS)
Cambios requieren	Reinstalar en servidor	Actualizar ficheros estáticos
Interactividad	Menor	Mayor
Carga inicial	Menor	Mayor

PREGUNTAS FRECUENTES DE EXAMEN

Arquitectura

1. **Desventaja 2 capas vs 3:** Cambios en Modelo requieren reinstalar en clientes
2. **JDBC se usa en:** Capa Acceso a Datos + Capa Lógica de Negocio

JDBC

3. **executeQuery():** Solo SELECT, siempre devuelve ResultSet
4. **Pool de conexiones:** close() NO cierra conexión real
5. **Parámetros PreparedStatement:** se numeran desde 1

Capa Modelo

6. **DAO no oculta:** tipo BD ni tecnología de acceso
7. **SQLException en DAO:** se relanza como RuntimeException
8. **Violación regla de negocio:** commit() + excepción checked

Pruebas

9. **Crear datos:** usar servicio si existe método adecuado
10. **Eliminar datos:** usar DAO si no hay método en servicio
11. **Excepciones en métodos privados:** capturar y relanzar como RuntimeException

JSON

12. **[{...}]**: Array con un objeto (no objeto directo)

REST

13. **Crear recurso:** POST a colección
14. **Modificar recurso:** PUT a /recurso/{id}
15. **Error validación:** 400 Bad Request
16. **Recurso no existe:** 404 Not Found
17. **Error interno:** 500 Internal Error
18. **Cache GET 200:** Sí se cachea
19. **Inscripción carrera:** POST /inscripciones

Thrift

20. **struct NO es exception:** No puede usarse como tipo de excepción
21. **Identificadores campos:** Positivos, únicos, no reutilizar si se elimina

RESPUESTAS PREGUNTAS TIPO EXAMEN

#	Respuesta
1	c) No permite cambios en Modelo sin reinstalar en clientes
2	d) Lógica de Negocio y Acceso a Datos
3	a) Corresponde a capa Acceso a Datos
4	b) close() en pool no cierra conexión real
5	b) Usar servicios de capa Modelo si existen
6	a) Array con objeto de 3 campos (string, array, número)

#	Respuesta
7	d) POST a /inscripciones
8	c) Todas correctas (cachear GET 200, invalidar tras DELETE 204)
9	c) struct NO puede usarse como tipo de excepción
10	b) Capturar y relanzar como RuntimeException